

TASK 12: #51632

- i. Create two instances, one in a public subnet and one in a private subnet
- ii. Host a website inside public web server
- iii. Host a database inside private database server
- iv. Connect to DB server only from its webserver, shouldn't have a public access to server even if the security group has open entries
- v. DB Server should have internet access. Try to update the software packages, patching, etc

i. Create the VPC, Subnets, and Networking

1. Create VPC

- VPC Console -> Create VPC
- Name: my-vpc
- CIDR: 10.0.0.0/16

2. Create two subnets

- Public subnet, e.g. 10.0.1.0/24
- Private subnet, e.g. 10.0.2.0/24

(Note: in this task, the actual assignment ended up as web server = 10.0.2.144 and DB server = 10.0.1.30. What matters is which subnet routes directly to the Internet Gateway vs. through NAT.)

3. Create and attach an Internet Gateway

- VPC -> Internet Gateways -> Create -> Attach to my-vpc

4. Create a NAT Gateway

- VPC -> NAT Gateways -> Create NAT Gateway
- Subnet: the public subnet
- Allocate a new Elastic IP for it

5. Configure route tables

Public route table (associated with public subnet):

Destination Target

10.0.0.0/16 local

0.0.0.0/0 Internet Gateway

Private route table (associated with private subnet):

Destination Target

10.0.0.0/16 local

0.0.0.0/0 NAT Gateway

Security Groups (Minimum Access)

web-sg (public/web instance)

Type Port Source

SSH 22 Your IP only

HTTP 80 0.0.0.0/0

HTTPS 443 0.0.0.0/0 (optional)

db-sg (private/DB instance)

Type Port Source

MySQL/Aurora 3306 web-sg (security group reference, not a CIDR)

SSH 22 web-sg (so the DB server is only SSH-able from the web server)

Key point: no rule on db-sg should ever reference 0.0.0.0/0 or a public CIDR. Combined with the DB instance having no public IP at all, this makes the database structurally unreachable from the internet, regardless of any security group mistake.

ii. Launch and Configure the Public Web Server

1. Launch instance

- AMI: Amazon Linux 2023
- Subnet: public subnet
- Auto-assign public IP: Enable
- Security group: web-sg

2. Install httpd, PHP, and extensions

```
sudo dnf install -y httpd php php-mysqlnd
```

```
sudo systemctl start httpd
sudo systemctl enable httpd
```

3. Download WordPress into a user directory

```
cd /home/ec2-user
wget https://wordpress.org/latest.tar.gz
tar -xzf latest.tar.gz
sudo chown -R ec2-user:ec2-user /home/ec2-user/wordpress
```

4. Point httpd's DocumentRoot to it

```
sudo vi /etc/httpd/conf/httpd.conf
```

Change:

```
DocumentRoot "/var/www/html"
```

to:

```
DocumentRoot "/home/ec2-user/wordpress"
```

Update the matching directory block:

```
<Directory "/home/ec2-user/wordpress">
    AllowOverride All
    Require all granted
</Directory>
```

5. Fix permissions and SELinux context

```
sudo chmod 711 /home/ec2-user
sudo semanage fcontext -a -t httpd_sys_content_t "/home/ec2-user/wordpress(/.*)?"
sudo setsebool -P httpd_can_network_connect_db 1
```

6. Restart httpd

```
sudo systemctl restart httpd
```

iii. Launch and Configure the Private Database Server

1. Launch instance

- AMI: Amazon Linux 2023
- Subnet: private subnet

- Auto-assign public IP: Disable
- Security group: db-sg

2. Access it (no public IP, so hop through the web server)

First :

Copy the private key from the and paste it in the public server

```
Sudo vi test.pem  
  
chmod 400 webserver-key.pem  
  
ssh -i test.pem ec2-user@<DB-server-private-IP>
```

(Better practice long-term: use ssh -A agent forwarding instead of copying the key file onto the web server.)

3. Install and start MariaDB

```
sudo dnf update -y  
  
sudo dnf install -y mariadb105-server  
  
sudo systemctl start mariadb  
  
sudo systemctl enable mariadb
```

4. Log in to MariaDB

On a fresh install, try:

```
sudo mysql
```

(This uses socket authentication and usually needs no password.)

Set a root password and secure the install:

```
sudo mysql_secure_installation
```

5. Create the WordPress database and a restricted user

```
CREATE DATABASE wordpress_db;  
  
CREATE USER 'wpuser'@'<web-server-subnet-CIDR-or-IP>' IDENTIFIED BY  
'StrongPassword123!';  
  
GRANT ALL PRIVILEGES ON wordpress_db.* TO 'wpuser'@'<web-server-subnet-CIDR-  
or-IP>';  
  
FLUSH PRIVILEGES;  
  
EXIT;
```

Example, if the web server's subnet is 10.0.2.0/24:

```
CREATE USER 'wpuser'@'10.0.2.%' IDENTIFIED BY 'StrongPassword123!';
```

iv. Connect to the DB Only From the Web Server

1. Install a MySQL/MariaDB client on the web server

```
sudo dnf install -y mariadb
```

2. Test the connection from the web server

```
mysql -h <DB-server-private-IP> -u wpuser -p wordpress_db
```

Enter the password when prompted — the terminal shows nothing at all while typing (no dots, no asterisks); this is normal, just type it and press Enter.

Expected outcomes:

Result	Meaning
MariaDB [wordpress_db]> prompt	Success — connection, credentials, and privileges are correct
ERROR 1045: Access denied	Wrong password, or the user's allowed host doesn't match where the connection is coming from
ERROR 1049: Unknown database	Database name typo (e.g., wordpress vs wordpress_db)
Hangs / times out (need Ctrl+C to escape)	Network-level block — see Troubleshooting below

3. Confirm the DB is NOT reachable from anywhere else

From your laptop (or any machine outside the VPC), attempting the same command should hang/fail — this confirms the private subnet + security group design is working as intended.

Connect WordPress to the Private DB Server

1. Configure wp-config.php on the web server

```
sudo cd /home/ec2-user/wordpress
sudo cp wp-config-sample.php wp-config.php
sudo vi wp-config.php
define( 'DB_NAME', 'wordpress_db' );
```

```
define( 'DB_USER', 'wpuser' );  
  
define( 'DB_PASSWORD', 'StrongPassword123!' );  
  
define( 'DB_HOST', '<DB-server-private-IP>' );
```

2. Restart httpd and load the site

bash

sudo systemctl restart httpd

Visit:

http://<web-server-public-IP>

This should show the WordPress installation wizard. Complete Site Title, admin username/password, email -> Install WordPress, then log in at /wp-admin.

OUTPUT:



